

Accounts App API Documentation - Complete Reference

Base URL

/api/accounts/

Authentication Notes

- Registration endpoints are public (no authentication required)
 - Token refresh endpoint requires valid refresh token
 - No authentication needed for login
-

Endpoint 1: User Registration (Regular User)

POST /api/accounts/user/register/

Allowed Access: Public (no authentication required)

Request Body:

json

```
{  
  "first_name": "Ali",  
  "last_name": "Ahmadi",  
  "phone": "09123456789",  
  "email": "ali.ahmadi@example.com",  
  "password": "StrongP@ssw0rd"  
}
```

Field Descriptions:

- first_name (string, required): User's first name
- last_name (string, required): User's last name
- phone (string, required): Valid mobile number (unique)
- email (string, required): Valid email address (unique)
- password (string, required): User's password (will be hashed)

Validation Rules:

- Phone number must be unique across all accounts
- Email must be unique across all accounts
- Password is write-only (not returned in response)

Response (201 Created):

json

```
{
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 5,
    "first_name": "Ali",
    "last_name": "Ahmadi",
    "phone": "09123456789",
    "email": "ali.ahmadi@example.com",
    "role": "user"
  }
}
```

Response Fields:

- refresh: Refresh token (long-lived, for obtaining new access tokens)
- access: Access token (short-lived, for API authentication)
- user: User account information
 - id: Unique user identifier
 - first_name, last_name, phone, email: User details
 - role: Always "user" for this endpoint

Error Responses:

- **400 Bad Request:**
 - "This phone has already registered"
 - "This email has already registered"
 - Missing required fields

Endpoint 2: Owner Registration (Company Owner)

POST /api/accounts/owner/register/

Allowed Access: Public (no authentication required)

Request Body:

json

```
{  
  "first_name": "Mohammad",  
  "last_name": "Rezaei",  
  "phone": "09987654321",  
  "email": "mohammad.rezaei@company.com",  
  "password": "CompanyP@ss123",  
  "company_name": "Pars Travel Co."  
}
```

Field Descriptions:

- first_name (string, required): Owner's first name
- last_name (string, required): Owner's last name
- phone (string, required): Valid mobile number (unique)
- email (string, required): Valid email address (unique)
- password (string, required): Owner's password (will be hashed)
- company_name (string, required): Name of the company (must be unique)

Validation Rules:

- Phone number must be unique across all accounts
- Email must be unique across all accounts
- Company name must be unique across all companies
- Company name is required

Atomic Transaction:

- Account and Company are created in a single database transaction
- If either fails, both are rolled back

Response (201 Created):

json

```
{
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "owner": {
    "id": 3,
    "first_name": "Mohammad",
    "last_name": "Rezaei",
    "phone": "09987654321",
    "email": "mohammad.rezaei@company.com",
    "role": "owner"
  }
}
```

Response Fields:

- refresh: Refresh token for the owner
- access: Access token for the owner
- owner: Owner account information
 - id, first_name, last_name, phone, email, role (always "owner")

Note: The created company is automatically linked to the owner, but company details are not returned in this response.

Error Responses:

- **400 Bad Request:**
 - "This phone has already registered"
 - "This email has already registered"
 - "company name must be entered"
 - "company name must be unique"
 - "Registration failed due to an error: {error_details}"

Endpoint 3: Account Login

POST /api/accounts/account/login

Allowed Access: Public (no authentication required)

Request Body:

json

```
{  
  "phone": "09123456789",  
  "password": "StrongP@ssw0rd"  
}
```

Field Descriptions:

- phone (string, required): Registered phone number
- password (string, required): Account password

Validation Rules:

- Account must exist with the given phone number
- Account must be active (is_active = True)
- Password must match

Response (200 OK):

json

```
{
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "account": {
    "id": 5,
    "first_name": "Ali",
    "last_name": "Ahmadi",
    "role": "user"
  }
}
```

Response Fields:

- access: JWT access token (include in Authorization header)
- refresh: JWT refresh token (for obtaining new access tokens)
- account: Basic account information
 - id: Account identifier
 - first_name, last_name: Account holder's name
 - role: "user" or "owner"

Error Responses:

- **400 Bad Request:** "Invalid credentials" (generic message for security)
- **401 Unauthorized:** Account inactive or invalid credentials

Endpoint 4: Refresh Access Token

POST /api/accounts/refresh-token/

Allowed Access: Requires valid refresh token

Request Body:

```
json
{
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Field Descriptions:

- refresh (string, required): The refresh token obtained from login or registration

Response (200 OK):

```
json
{
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Response Fields:

- access: New access token (use for subsequent API requests)

Error Responses:

- **401 Unauthorized:** Invalid or expired refresh token

Usage:

When your access token expires, use this endpoint to obtain a new one without re-authenticating.

Account Roles Summary

Role	Identifier	Can Create Bookings?	Can Manage Company?
User	"user"	 Yes	 No
Owner	"owner"	 No	 Yes (via Companies app)

Role Characteristics:

- **User:** Regular customer who can book trips
 - **Owner:** Company representative who manages trips and bookings for their company
-

Token Usage

After obtaining access token, include it in all authenticated requests:

Authorization Header:

text

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Token Lifetimes:

- Access Token: Short-lived (typically 5 minutes to 24 hours, configured in settings)
- Refresh Token: Longer-lived (typically 7-30 days)

Token Refresh Flow:

text

1. Access token expires
 2. Client sends refresh token to /refresh-token/
 3. Receive new access token
 4. Continue API calls with new token
-

Complete API Endpoints Summary

Method	Endpoint	Authentication	Role	Description
POST	/api/accounts/user/register/	None	Public	Register regular user

Method	Endpoint	Authentication	Role	Description
POST	/api/accounts/owner/register/	None	Public	Register company owner
POST	/api/accounts/account/login	None	Public	Login and get tokens
POST	/api/accounts/refresh-token/	Refresh Token	All	Refresh access token

HTTP Status Codes Summary

Status Code	Meaning	When Used
200 OK	Success	Successful login, token refresh
201 Created	Resource created	Successful user/owner registration
400 Bad Request	Client error	Validation failure, duplicate data
401 Unauthorized	Authentication failed	Invalid credentials, expired token

Example API Calls (cURL)

Register Regular User

```
bash
```

```
curl -X POST http://localhost:8000/api/accounts/user/register/ \
```

```
-H "Content-Type: application/json" \
```

```
-d '{
  "first_name": "Ali",
  "last_name": "Ahmadi",
  "phone": "09123456789",
  "email": "ali@example.com",
  "password": "StrongP@ss123"
}'
```

Register Company Owner

bash

```
curl -X POST http://localhost:8000/api/accounts/owner/register/ \
-H "Content-Type: application/json" \
-d '{
  "first_name": "Mohammad",
  "last_name": "Rezaei",
  "phone": "09987654321",
  "email": "mohammad@travelco.com",
  "password": "CompanyP@ss123",
  "company_name": "Pars Travel Co."
}'
```

Login

bash

```
curl -X POST http://localhost:8000/api/accounts/account/login \
-H "Content-Type: application/json" \
-d '{
  "phone": "09123456789",
  "password": "StrongP@ss123"
```

```
}'
```

Refresh Access Token

bash

```
curl -X POST http://localhost:8000/api/accounts/refresh-token/ \
```

```
-H "Content-Type: application/json" \
```

```
-d '{
```

```
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

```
}'
```

Important Security Notes

1. **Password Storage:** Passwords are hashed using Django's built-in `create_user` method (PBKDF2 algorithm)
2. **Phone Uniqueness:** Phone numbers are unique identifiers across all account types
3. **Email Uniqueness:** Email addresses must be unique in the system
4. **Company Name Uniqueness:** Company names are enforced to be unique to prevent duplicates
5. **Generic Error Messages:** Login returns "Invalid credentials" (not "user not found" or "wrong password") for security
6. **Token Security:**
 - Store tokens securely (e.g., `HttpOnly` cookies or secure storage)
 - Never expose tokens in URLs or logs
 - Use HTTPS in production
7. **Account Status:** Only active accounts (`is_active=True`) can log in

Data Model Relationships

text

Account (User/OWNER)

↓

└─ Company (if role = OWNER)

└─ Trips

└─ Bookings (via trips)

Key Relationships:

- Each Owner has exactly one Company
- Each Company has exactly one Owner
- Company owners cannot register another company with the same phone/email

Version Information

- **API Version:** 1.0
- **Last Updated:** May 2026
- **Authentication:** JWT (JSON Web Tokens)
- **Token Refresh Supported:** Yes